

Quiz 1: Fundamentos de Desarrollo Web y Arquitectura RESTful

Curso: IF0009 - Desarrollo de Software IV

Docente: Mag. Jonathan Granados C.

Puntuación Total: 25 Puntos

Tiempo Límite Recomendado: 30 minutos

Instrucciones: Lea detenidamente cada una de las siguientes preguntas de opción múltiple y seleccione la respuesta que considere correcta. Al finalizar la prueba, puede consultar el solucionario analítico ubicado al final del documento para verificar sus razonamientos.

■ Cuestionario de Evaluación (30 Minutos)

1. Arquitecturas Web: SPA vs MPA y Ciclo de Renderizado (5 Puntos)

Un equipo de arquitectura de software debe seleccionar la arquitectura adecuada para una aplicación web empresarial con alta interactividad. Se requiere reducir el tráfico de red general al navegar entre vistas y evitar recargas completas de la página (*full page reload*). ¿Cuál es la diferencia fundamental entre una Single-Page Application (SPA) y una Multi-Page Application (MPA) en términos de entrega de contenidos?

- ☐ A) La MPA descarga una sola vez la plantilla HTML/JS base y actualiza la interfaz mediante peticiones asíncronas de datos (JSON), mientras que la SPA solicita al servidor un nuevo documento HTML completo en cada interacción.
- ☐ B) La SPA procesa la lógica de renderizado visual en el navegador del cliente consumiendo APIs RESTful, mientras que la MPA delega el renderizado de cada vista HTML al servidor ante cada petición del usuario.
- ☐ C) La SPA no puede consumir servicios web de back-end debido a restricciones de seguridad del navegador, obligando al uso de arquitecturas monolíticas de renderizado en servidor.
- ☐ D) La MPA elimina el uso de hojas de estilo CSS y archivos JavaScript, transfiriendo únicamente etiquetas HTML nativas sin soporte para interactividad.

2. Semántica de Verbos HTTP, Idempotencia y Códigos de Estado (5 Puntos)

En el diseño de una API RESTful para la gestión de productos en un catálogo de comercio electrónico, se requiere implementar un endpoint para crear un nuevo producto y otro para reemplazar de forma integral la información de un producto existente con ID conocido. ¿Cuál es la especificación correcta de verbos HTTP, propiedad de idempotencia y códigos de respuesta esperados?

- ☐ A) Usar `POST` para creación (retornando `201 Created`, no idempotente) y `PUT` para reemplazo total (retornando `200 OK` o `204 No Content`, idempotente).
- ☐ B) Usar `GET` para creación (retornando `200 OK`, idempotente) y `POST` para reemplazo total (retornando `201 Created`, no idempotente).
- ☐ C) Usar `PUT` para creación (retornando `404 Not Found`) y `PATCH` para reemplazo total (retornando `500 Internal Server Error`).
- ☐ D) Usar `POST` tanto para creación como para reemplazo total, garantizando que ambas operaciones sean estrictamente idempotentes por estándar.

3. Arquitectura en Capas y Patrón de Inyección de Dependencias (5 Puntos)

En un proyecto de back-end desarrollado con Spring Boot, se aplica una arquitectura desacoplada estructurada en capas (`controller`, `business/service`, `data/repository`). ¿Cuál es la responsabilidad estricta del Controlador REST (`@RestController`) y cuál es la buena práctica recomendada para inyectar la capa de negocio (`@Service`)?

- ☐ A) El Controlador debe ejecutar directamente las consultas SQL a la base de datos e inyectar el Servicio mediante modificación directa de variables globales estáticas.
- ☐ B) El Controlador debe gestionar únicamente la recepción de solicitudes HTTP, mapeo de parámetros/DTOs y retorno de respuestas HTTP, inyectando el Servicio a través del constructor de la clase.

- [] C) El Controlador debe contener la lógica de validación de reglas de negocio pesadas y crear instancias del Servicio manualmente utilizando el operador `new`.
- [] D) El Controlador actúa como repositorio de persistencia JPA y debe estar anotado obligatoriamente con `@Entity` y `@Table`.

4. Ejercicio de Rastreo de Código (Code Tracing) - Rutas y Parámetros en Controller REST (5 Puntos)

Analice detenidamente la siguiente implementación de un controlador en Spring Boot:

```
@RestController
@RequestMapping("/api/v1/estudiantes")
public class EstudianteController {

    private final EstudianteService estudianteService;

    public EstudianteController(EstudianteService estudianteService) {
        this.estudianteService = estudianteService;
    }

    @GetMapping("/{id}")
    public ResponseEntity<EstudianteDto> obtenerPorId(
        @PathVariable("id") Long id,
        @RequestParam(value = "incluirDetalles",
            defaultValue = "false") boolean incluirDetalles) {

        EstudianteDto dto = estudianteService
            .buscarEstudiante(id, incluirDetalles);
        return ResponseEntity.ok(dto);
    }
}
```

Suponga que un cliente efectúa la siguiente solicitud HTTP:

`GET /api/v1/estudiantes/105?incluirDetalles=true`

¿Qué valores exactos pasarán como argumentos al método `buscarEstudiante` del servicio durante la ejecución del controlador?

- ☐ A) `id = null` e `incluirDetalles = false`, debido a que los parámetros de ruta requieren `@RequestParam`.
- ☐ B) `id = 105L` e `incluirDetalles = true`, extraídos correctamente de la variable de plantilla URI y del parámetro de consulta (*query param*).
- ☐ C) `id = "105"` e `incluirDetalles = "true"`, lanzando una excepción `ClassCastException` por no usar el tipo `String`.
- ☐ D) `id = 0L` e `incluirDetalles = false`, ignorando el parámetro de la URI por falta de la anotación `@RequestBody`.

5. Contratos de API RESTful y Documentación con OpenAPI / Swagger (5 Puntos)

Al construir sistemas distribuidos desacoplados, es fundamental establecer un "contrato de API" claro y legible entre los desarrolladores de back-end y front-end. ¿Cuál es el propósito principal de integrar la especificación OpenAPI 3.0 (mediante herramientas como Swagger UI) en una API RESTful?

- ☐ A) Compilar el código Java directamente a componentes nativos del navegador web sin necesidad de usar JavaScript.
- ☐ B) Documentar e interactuar dinámicamente con los endpoints, parámetros, estructuras DTO y respuestas HTTP de la API sin necesidad de acceder al código fuente.
- ☐ C) Reemplazar la base de datos relacional del sistema por una interfaz gráfica estática de consulta temporal.
- ☐ D) Bloquear la ejecución de peticiones HTTP que no provengan del servidor local Tomcat.

■ Solucionario Analítico y Justificaciones Pedagógicas

Pregunta 1: Arquitecturas Web: SPA vs MPA y Ciclo de Renderizado

- **Respuesta Correcta: B**
- **Justificación Analítica:**
- En una **Single-Page Application (SPA)**, el navegador descarga el cascarón de la aplicación (HTML básico y bundles de JavaScript). Toda la lógica de presentación y cambios de vista ocurren dinámicamente en el cliente (Client-Side Rendering) consumiendo datos JSON de una API REST. En contraste, una **Multi-Page Application (MPA)** procesa el renderizado de cada página HTML completa en el servidor (Server-Side Rendering) ante cada navegación del usuario.
- **Análisis de Distractores:**
- **A es incorrecta:** Invierte los conceptos (asigna el comportamiento de la SPA a la MPA y viceversa).
- **C es incorrecta:** Las SPAs consumen rutinariamente servicios web RESTful mediante Fetch API o HttpClient.
- **D es incorrecta:** Las MPAs tradicionales emplean CSS y JavaScript activamente; la diferencia radica en el ciclo de vida de la página y el renderizado en servidor.

Pregunta 2: Semántica de Verbos HTTP, Idempotencia y Códigos de Estado

- **Respuesta Correcta: A**
- **Justificación Analítica:**
- **POST** es la acción estándar para la creación de recursos (no es idempotente, ya que ejecutar múltiples peticiones idénticas crea múltiples recursos distintos) y debe retornar el código **201 Created**. **PUT** se utiliza para la sustitución completa de un recurso conocido o su creación si se define la URI (es idempotente, pues repeticiones del mismo payload producen el mismo estado final) y retorna **200 OK** con cuerpo o **204 No Content** sin cuerpo.
- **Análisis de Distractores:**
- **B es incorrecta:** **GET** es exclusivo para lectura segura y nunca debe alterar el estado del servidor ni crear entidades.
- **C es incorrecta:** **404** y **500** son códigos de error, no de éxito en la creación/actualización.
- **D es incorrecta:** **POST** no es una operación idempotente por definición técnica del estándar HTTP RFC 7231.

Pregunta 3: Arquitectura en Capas y Patrón de Inyección de Dependencias

- **Respuesta Correcta: B**
- **Justificación Analítica:**
 - En la arquitectura desacoplada de Spring Boot, el `@RestController` tiene la única responsabilidad de actuar como interfaz de transporte HTTP. La buena práctica de diseño (aprobada por Spring) exige la inyección de dependencias a través del **constructor** de la clase (favoreciendo la inmutabilidad con campos `final` y facilitando las pruebas unitarias sin requerir reflexión).
- **Análisis de Distractores:**
 - **A es incorrecta:** Acceder a la base de datos desde el controlador viola la separación de capas y acopla la persistencia al transporte.
 - **C es incorrecta:** Instanciar servicios manualmente con `new` destruye la gestión del contenedor IoC (Inversion of Control) de Spring.
 - **D es incorrecta:** Las anotaciones `@Entity` y `@Table` corresponden exclusivamente a clases del modelo de datos/persistencia JPA.

Pregunta 4: Ejercicio de Rastreo de Código (Code Tracing) - Rutas y Parámetros en Controller REST

- **Respuesta Correcta: B**
- **Justificación Analítica:**
 - La anotación `@GetMapping("/{id}")` vincula el segmento de ruta `105` al argumento `Long id` anotado con `@PathVariable("id")`. Por su parte, `@RequestParam(value = "incluirDetalles", defaultValue = "false")` intercepta el parámetro de consulta `?incluirDetalles=true` y convierte la cadena `"true"` al tipo primitivo boolean `true`. En consecuencia, los valores evaluados son `105L` y `true`.
- **Análisis de Distractores:**

- **A es incorrecta:** `@PathVariable` procesa correctamente variables de plantilla en la URI; no requiere `@RequestParam`.
 - **C es incorrecta:** Spring Boot realiza la conversión automática de tipos de datos de cadenas a tipos primitivos y envoltorios (`Long`, `boolean`).
 - **D es incorrecta:** `@RequestBody` se usa para leer el cuerpo de la petición (JSON) en verbos como POST o PUT, no para variables de ruta en peticiones GET.
-

Pregunta 5: Contratos de API RESTful y Documentación con OpenAPI / Swagger

- **Respuesta Correcta: B**
- **Justificación Analítica:**
- OpenAPI 3.0 proporciona una especificación estandarizada y agnóstica al lenguaje para describir APIs RESTful. Herramientas como Swagger UI generan un portal web interactivo que permite explorar endpoints, visualizar modelos DTO y probar peticiones en tiempo real sin requerir acceso al código fuente del back-end.
- **Análisis de Distractores:**
- **A es incorrecta:** Swagger/OpenAPI no transpila Java a JavaScript ni genera código ejecutable para el cliente.
- **C es incorrecta:** OpenAPI no reemplaza el motor de persistencia ni la base de datos relacional.
- **D es incorrecta:** OpenAPI no actúa como un filtro o firewall de restricciones de origen ni bloquea peticiones HTTP.